

Faculty of Engineering and Technology
Electrical and Computer Engineering Department

APPLIED CRYPTOGRAPHY
ENCS 4320

Course Project:

SecureChannel — A Secure Communication Application

Prepared by:
Sarah Hassouneh **1210068**
Islam Zayed **1230007**

Instructor: Dr. Mohammad Hussein

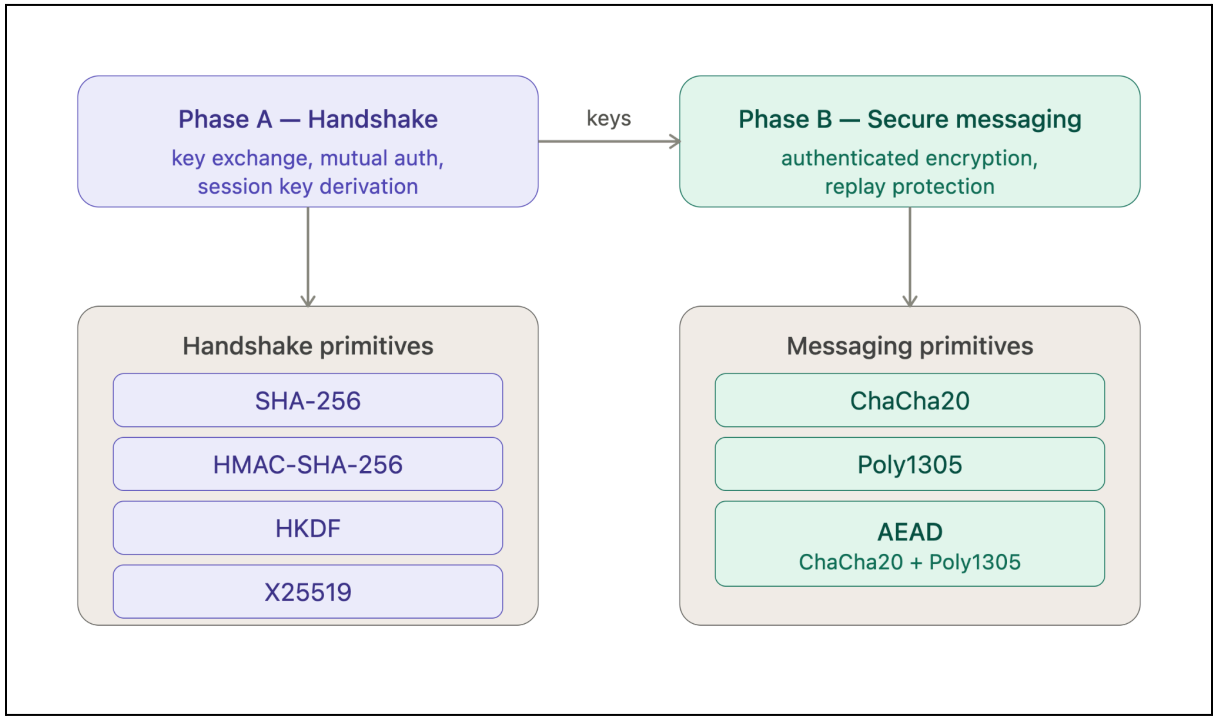
Section: 2
Date: June 27, 2025

System Overview

The system we have built is basically a SecureChannel which mimics a secure, end-to-end encrypted chat application built entirely from scratch in Python. We assume that there are two parties, a client and a server that communicate with each other over a TCP connection. The objective of this secure channel is to keep the communication process confidential, authenticated, and tamper-proof.

We implement the system into 2 phase, the first phase is the handshake phase that use 4 algorithms: SHA-256, HMAC, HKDF and X25519 to establish the connection and the handshake protocol. The output of this phase is the session keys, that are then used for the second phase in the Secure Messaging. The secure messaging using the ChaCha20, Poly1305, and the AEAD construction to exchange messages between the server and client.

Every primitive was implemented from scratch following its official specification (FIPS 180-4, RFC 2104, RFC 5869, RFC 7748, RFC 8439) and tested against published test vectors from those same specifications.



Handshake Phase:

The handshake phase's main purpose is to set up a secure connection, and generate secure session keys between the client and the server.

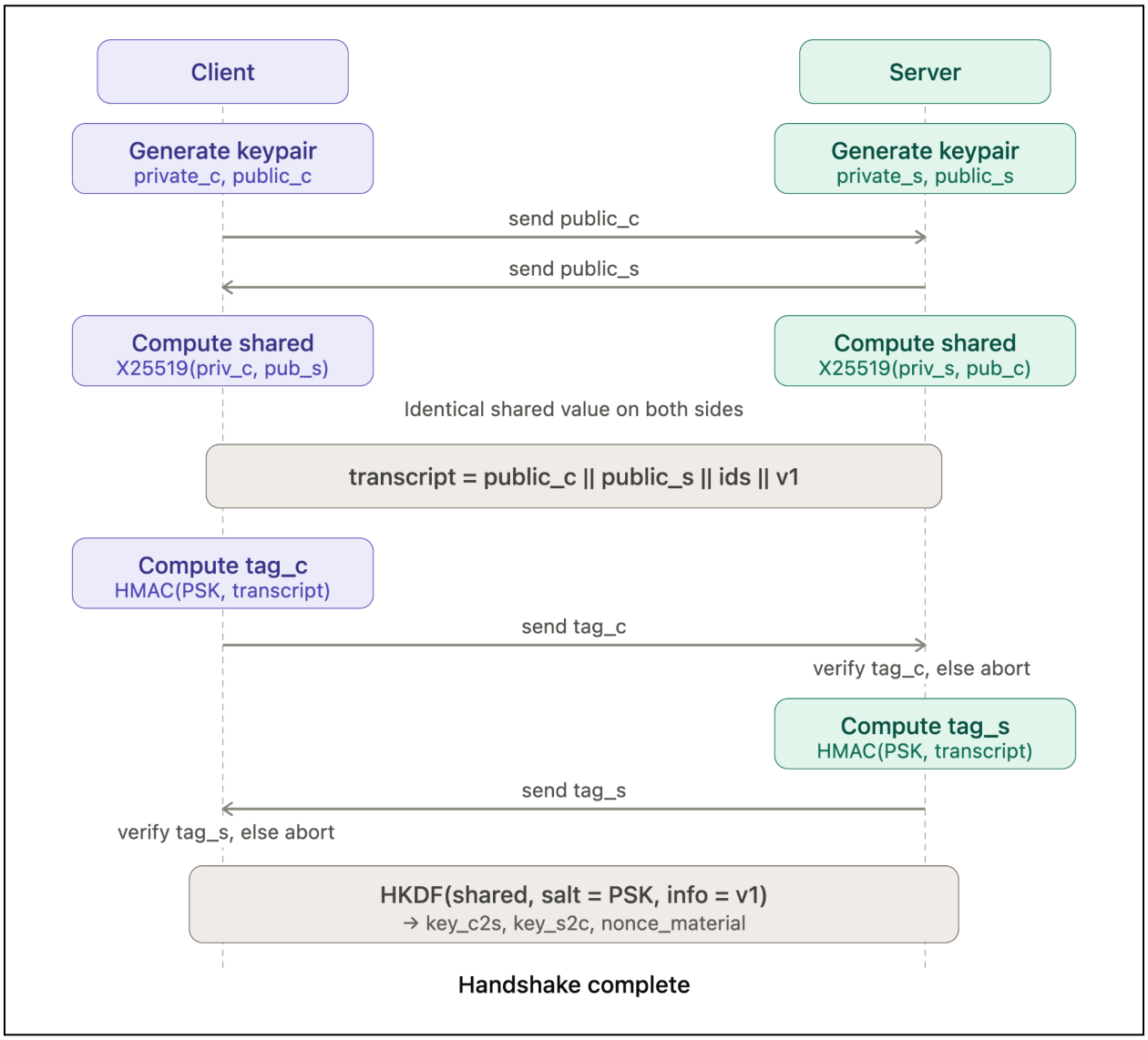
Security Goals during Handshake:

We define the intended security goals of the handshake phase as follows:

- **Forward secrecy:** Through the compromise of a long-term key (the PSK), the system does not allow an attacker to decrypt past session traffic, since each session's keys are derived from ephemeral X25519 key pairs that are generated fresh per session, used once, and never stored or transmitted. Once a session ends, the keys needed to decrypt it no longer exist anywhere.
- **Mutual authentication:** both parties prove they know the pre-shared key (PSK) before trusting the connection.
- **Transcript binding:** the exchanged public keys are cryptographically tied to both parties' identities, preventing man-in-the-middle substitution.

The diagram below explains how the handshake phase does that. It starts by each of the client and server generating both a private and a public key, they then interact with each other and exchange the public keys with one another. Following that, each computes a shared secret from its own private key, and the public key of the other party. This shared key is generated with X25519, which is a type of elliptic curve cryptography that uses a curve over a 255-bit prime field, chosen because it is fast and resistant to known attacks on the discrete logarithm problem. After that, both server and client generate an identical transcript of the public keys, their IDs (which is basically just their names "client" and "server") and a version. Here we used "v1".

For integrity purposes, we further need to compute a tag that is generated by the HMAC algorithm, which uses a PSK, which is a pre-shared key, and the generated transcript, these tags are shared and verified on each side. Following that, the HKDF algorithm uses the shared secret, a salt (which is the PSK, mixed in to bind the derived keys to it), and info, which we chose to be the version name. By the end we have 2 keys: key_c2s, which is used when the client sends



messages to the server, and key_s2c, which is used when the server sends messages to the client, and additionally a nonce_material, which is combined with the message sequence number to produce a unique nonce for each message.

Key Exchange:

Each party generates an ephemeral X25519 keypair and sends its public key to the other. Each side then computes shared_secret = X25519(my_private, peer_public), arriving independently at the same value without it ever being transmitted. X25519 ensures the shared secret never crosses the network, even though the public keys do.

Authentication:

Both sides build an identical transcript: public_c || public_s || "client" || "server" || "v1". Each computes HMAC(PSK, transcript) and sends the tag to the other side. HMAC proves both parties know the PSK, and binding the transcript to both public keys and identities prevents an attacker from substituting a different key pair without detection. If the received tag does not match the locally computed one, the handshake aborts with an authentication error, and no session keys are produced.

Key Derivation:

The shared secret produced by X25519 is not directly suitable for use as an encryption key, since it is a single mathematically structured value rather than uniformly random bytes. To address this, the shared secret is passed into HKDF along with the PSK as salt and a version string as info, producing a longer stream of properly randomized key material. In total, in our system the length of this key material is 76 bytes in total: 32 bytes for each of the two directional keys, plus 12 bytes for the nonce material.

This output is then split into three pieces: a client-to-server key (key_c2s), a server-to-client key (key_s2c), and nonce_material. **Using two separate directional keys,** rather than one shared key for both directions, is important because ChaCha20 is a stream cipher reusing the same key and nonce in two different messages produces an identical keystream, which can leak information about both messages. Since both the client and server independently track their own sequence numbers starting from the same value, a shared key would risk this exact collision. Giving each direction its own key eliminates this risk entirely, even if both sides happen to use the same sequence number at the same time.

Testing :

All the algorithms and primitives have been tested thoroughly, through the pucblished test vectors provided in the official documentation as shown below. All has been verified to pass all the test vectors and work smoothly. We employed pytest, a Python test suite, to automate verification: each algorithm's test file loads its corresponding vectors from a JSON file, runs the implementation against every vector, and asserts the output matches the expected result exactly.

This allowed every primitive (SHA-256, HMAC, HKDF, X25519, ChaCha20, Poly1305, and AEAD) to be verified independently and repeatably, both during development and after any later changes, ensuring no regressions were introduced as the implementation evolved.

Algorithms Used :

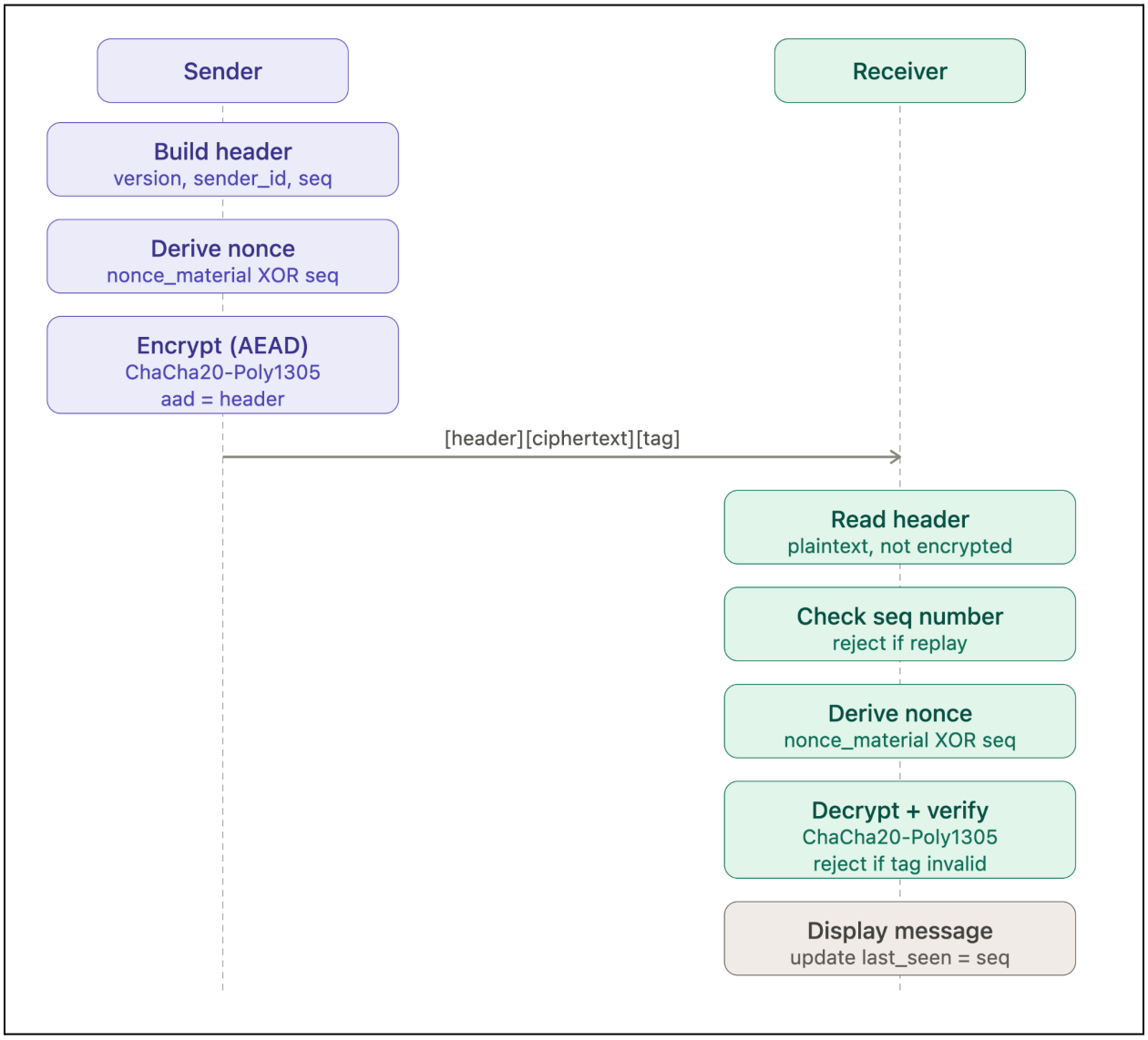
The handshake phase relies on four algorithms, each implemented from its official specification:

Algorithm	RFC / Spec	Input and Output	General Idea
SHA-256	FIPS 180-4	arbitrary-length message → 32-byte (256-bit) digest	This is a one-way hash function that HMAC and HKDF are both built on top of. The algorithm uses bitwise operations (AND, XOR, rotations) on 32-bit words.
HMAC-SHA-256	RFC 2104 (algorithm), RFC 4231 (test vectors)	key (variable length) + message (variable length) → 32-byte tag	This algorithm provides a keyed hash that proves the sender knows a secret (the PSK), used to authenticate the handshake. It uses byte operations, mainly XOR with padding constants, then two passes through SHA-256.
HKDF	RFC 5869	shared secret (32 bytes) + salt + info → derived key material (any length, 76 bytes here)	Built on repeated HMAC-SHA-256 calls (extract, then expand). Its main use is to stretch the raw shared secret into multiple independent, properly randomized keys.
X25519	*RFC 7748	private key (32 bytes) + peer's public key (32 bytes) → shared secret (32 bytes)	The idea from elliptic curve cryptography is to let both sides agree on a shared secret without ever sending it over the network. This version uses Elliptic curve scalar multiplication (Curve25519, over a 255-bit prime field).

*Also used Kleppmann tutorial to understand the x25519 eclectic curve cryptography.

Secure Messaging Phase (AEAD Channel):

After the handshake phase establishes a shared secret and derives the session keys, all application data is protected using the ChaCha20 - Poly1305 authenticated encryption algorithm defined in RFC 8439. This AEAD (Authenticated Encryption with Associated Data) construction provides confidentiality, integrity, and authenticity for every transmitted message. The channel operates over an insecure TCP connection and assumes the presence of an active network attacker capable of eavesdropping, modifying or replaying messages.



Security Goals:

The secure message layer guarantees the following:

- **Confidentiality:** Messages cannot be read and understood without the session key.
- **Integrity:** Any modification of ciphertext or metadata is detected.
- **Authentication:** Only a party with valid session keys can produce valid messages.
- **Replay Protection:** Previously captured messages cannot be resent successfully.
- **Header Protection:** Metadata is authenticated via AEAD associated data (AAD).

Message Format:

Each transmitted packet is structured as follows:

```
+----------------------+----------------+----------------------+
| Header (16 bytes)    | Ciphertext     | Poly1305 Tag (16 B)  |
+----------------------+----------------+----------------------+
```

The header is transmitted in plaintext because it contains information required to process the message before decryption. The header contains the following fields:

Field	Size	Description
Protocol Version	1 byte	Protocol version identifier
Message Type	1 byte	Identifies the message as a chat message
Sender ID	1 byte	Client (0) or Server (1)
Reserved	1 byte	Reserved for future extensions (must be 0)
Sequence Number	8 bytes	Monotonically increasing message counter
Plaintext Length	4 bytes	Length of the encrypted payload

The header is supplied to the AED algorithm as Additional Authenticated Data (AAD), ensuring that any modification of these fields invalidates the authentication tag.

Key Schedule:

The secure messaging layer does not generate keys directly. Instead, it uses the session keys established during the handshake phase.

HKDF derives three values from the shared Diffie-Hellman secret:

- Client-to-server encryption key (`key_c2s`)
- Server-to-client encryption key (`key_s2c`)
- Shared nonce material

Each communication direction therefore uses an independent encryption key, preventing keystream reuse between client and server.

Nonce Construction:

ChaCha20 - Poly1305 requires a unique 96-bit nonce for every encrypted message. Rather than transmitting nonces explicitly, both peers derive them independently from the shared nonce material and the message sequence number. For each transmitted message, the nonce is generated as follows:

nonce = nonce_material XOR sequence_number

where the sequence number is encoded as a 12 byte little-endian integer before performing the XOR operation. Identical nonces are capable of being generated independently without increasing packet size because both parties maintain synchronized sequence numbers. Since every sequence number is unique, nonce reuse cannot occur during a session.

ChaCha20 Stream Cipher (Encryption):

ChaCha20 operates as a stream cipher, utilizing the following to generate a pseudorandom keystream:

- ➢ A 256-bit secret key
- ➢ 96-bit nonce
- ➢ 32-bit block counter

- ❖ **Symmetric Operation:** Encryption is executed by performing a bitwise XOR operation between the generated keystream and the plaintext payload. Given XOR's self-inverse nature, decryption follows the identical mathematical operation.

- **Counter Management:** Compliant with RFC 8439, block counter 0 is strictly reserved for the derivation of the one-time Poly1305 authentication key. Payload encryption systematically commences at block counter 1.

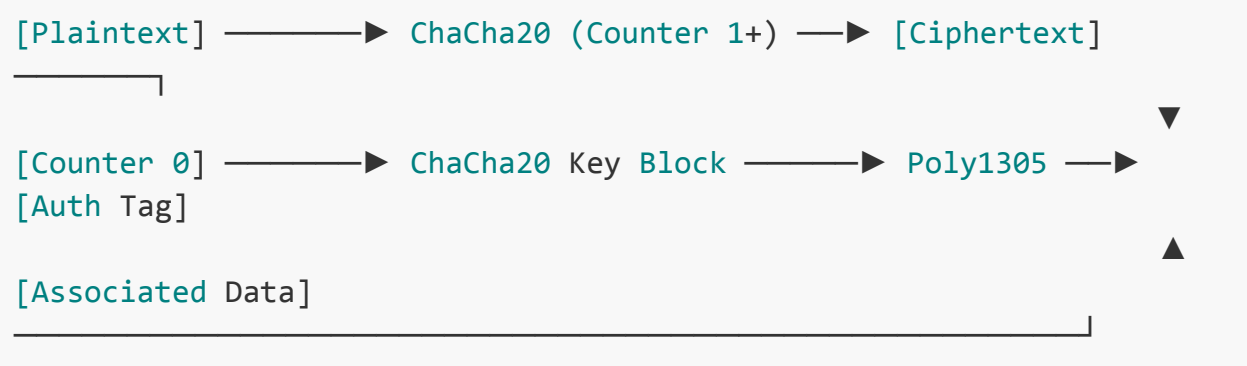
Poly1305 Message Authentication:

Poly1305 is a cryptographically secure, one-time Message Authentication Code (MAC) that outputs a 128-bit authentication tag.

- **Key Derivation**: To ensure strict single-use (ephemeral) security, the 256-bit Poly1305 subkey is generated dynamically by processing the first block (counter 0) of the ChaCha20 keystream.
- **Tag Verification:** The tag is computed over both the ciphertext and any associated unencrypted data. Upon receipt, the destination node independently computes the expected MAC and compares it against the received tag using a constant-time comparison algorithm (`hmac.compare_digest`). If a mismatch occurs, the packet is immediately dropped, and the decryption routine is aborted to mitigate side-channel and padding oracle vectors

Authenticated Encryption with Associated Data (AEAD) Construction:

The ChaCha20 - Poly1305 pipeline couples encryption and authentication into an atomic operation. The precise execution order for outbound packet processing is structured as follows:



1. **Subkey Generation:** Execute a ChaCha20 block operation with counter 0 to derive the ephemeral Poly1305 MAC key.

2. **Payload Encryption:** Encrypt the plaintext using a ChaCha20 beginning at counter 1 to produce the ciphertext.
3. **MAC Input Serialization:** Construct the Poly1305 input buffer by sequentially concatenating the following parameters:
    - Additional Authenticated Data (AAD)
    - Padding 1: Zero-byte padding to align the AAD to a 16-byte boundary
    - Ciphertext
    - Padding 2: Zero-byte padding to align the ciphertext to a 16-byte boundary
    - AAD Length: 64-bit unsigned integer (little-endian)
    - Ciphertext Length: 64-bit unsigned integer (little-endian)
4. **Tag Generation:** Compute the 16-byte Poly1305 MAC over the compiled input buffer

Sending

For every outgoing message, the sender performs the following steps:

1. Read plaintext from the user
2. Construct the protocol header
3. Generate the message nonce from the sequence number
4. Encrypt the plaintext using ChaCha20-Poly1305
5. Authenticate the header as Additional Authenticated Data
6. Transmit the packer consisting of the header, ciphertext, and authentication tag

Receiving (AEAD Decryption and Verification):

The receiving entity mirrors the exact pipeline described previously in AEAD encryption, validating the generated MAC prior to executing payload decryption. Any unauthorized manipulation of the ciphertext or the plaintext metadata within the AAD results in an immediate authentication failure. The exact operation performed by the receiver upon receiving a message may be described as follows:

1. Parse the unencrypted header
2. Validate the header fields
3. Verify that the sequence number has not been replayed:
    - Reject if `sequence_number <= last_seen_seq_number`

4. Recompute nonce using the same derivation rule
5. Verify the Poly1305 authentication tag before decryption
    - Reject message if tag is invalid
6. Decrypt ciphertext using ChaCha20 only after successful authentication
7. Display the recovered plaintext

This ensures no plaintext is released unless integrity is verified.

Replay Protection:

Replay protection is implemented using monotonically increasing sequence numbers. The receiver stores the largest valid sequence number previously processed. Every incoming packet must contain a strictly greater sequence number.

```
if sequence_number <= last_seen_seq_number:
     reject packet
```

Packets with duplicate or older sequence numbers are immediately discarded, preventing replay attacks and out-of-order packet acceptance.

Tamper Protection:

The protocol protects both the encrypted payload and the plaintext header using the AEAD construction. The authentication tag is computed over the following:

- ➢ The plaintext header (AAD)
- ➢ The ciphertext
- ➢ The required padding
- ➢ The lengths of the AAD and the ciphertext

Upon reception, the receiver independently recomputes the authentication tag and compares it against the received tag using a constant-time comparison (`hmac.compare_digest`). If the tags differ, the message is rejected without performing decryption. An attacker cannot modify metadata such as the sender identifier, protocol version, or sequence number without detection because the header is authenticated as AAD.

Verification and Test Methodology:

Test Coverage:

To ensure strict compliance and prevent implementation flaws, each cryptographic sub-component was validated independently against the official test vectors published in RFC 8439, Section 2.

- ChaCha20 Stream Cipher:
    - **Quarter Round Routine (RFC 8439 §2.1.1):** Verifies the basic Add-Rotate-XOR (ARX) mixing steps on a single vector column or diagonal using the official values
    - **Quarter Round on State (RFC 8439 §2.2.1):** Confirms that the quarter round correctly updates the specified words within the 16-word ChaCha20 state matrix
    - **Initial State Construction:** Validates the arrangement of the constants, key, block counter, and nonce into the initial 512-bit state.
    - **Block Function Execution (RFC 8439 §2.3.2):** Confirms the twenty rounds of ChaCha20 produce the expected 64-byte keystream block
    - **Encryption Function (RFC 8439 §2.4.2):** Verifies full stream encryption against the official ciphertext test vector and includes an additional empty-plaintext edge case
    - **One-Time Key Generation (RFC 8439 §2.6.2):**: Verifies that the ChaCha20 block function correctly derives the one-time Poly1305 key used by the AEAD construction

```
============================= test session starts ==============================
collecting ... collected 7 items

tests/test_chacha20.py::test_chacha20_vectors[RFC 8439 \u0622\xa72.1.1 Quarter Round Test] PASSED [ 14%]
tests/test_chacha20.py::test_chacha20_vectors[RFC 8439 \u0622\xa72.2.1 Quarter Round on State] PASSED [
28%]
tests/test_chacha20.py::test_chacha20_vectors[RFC 8439 \u0622\xa72.2.1 Initial State] PASSED [ 42%]
tests/test_chacha20.py::test_chacha20_vectors[RFC  8439  \u0622\xa72.3.2  Block  Function  Test]  PASSED  [
57%]
tests/test_chacha20.py::test_chacha20_vectors[RFC 8439 \u0622\xa72.4.2 Encryption Test] PASSED [ 71%]
tests/test_chacha20.py::test_chacha20_vectors[Empty plaintext] PASSED    [ 85%]
tests/test_chacha20.py::test_chacha20_vectors[RFC 8439 \u0622\xa72.6.2 Poly1305 One-Time Key Generation]
PASSED [100%]

============================== 7 passed in 0.04s ===============================
```

- Poly1305 MAC:
    - **MAC Generation (RFC 8439 §2.5.2):**: Validates the Poly1305 authentication tag using the official test vector
    - **Empty Message:** Confirms correct behavior when authenticating a zero-length message

```
============================= test session starts ==============================
collecting ... collected 2 items

tests/test_poly1305.py::test_poly1305_vectors[RFC 8439 Poly1305 Test Vector #1] PASSED [ 50%]
tests/test_poly1305.py::test_poly1305_vectors[RFC 8439 Poly1305 Empty Message] PASSED [100%]

============================== 2 passed in 0.03s ===============================
```

- AEAD:
    - Authenticated Encryption **(RFC 8439 §2.8.2)**: Validates both the generated ciphertext and the 128-bit authentication tag using the official AEAD test vector.
    - Authenticated Decryption **(RFC 8439 Appendix A.5)**: Verifies successful recovery of the original plaintext only when the authentication tag is valid, confirming the correctness of both verification and decryption.

```
============================= test session starts ==============================
collecting ... collected 2 items

tests/test_aead.py::test_aead_vectors[RFC 8439 AEAD Test Vector] PASSED  [ 50%]
tests/test_aead.py::test_aead_vectors[RFC 8439 Appendix A.5 AEAD Decryption Test] PASSED [100%]

============================== 2 passed in 0.07s ===============================
```

Passing the official RFC test vectors demonstrates that the implementation conforms to the published standard before integration into the secure messaging protocol.

Conclusion:

This project demonstrated the complete design and implementation of a secure communication system using modern cryptographic primitives implemented from their specifications. Rather than relying on external cryptographic libraries, each required component was developed in pure Python and validated against the corresponding RFC test vectors before being integrated into the final protocol. The resulting Secure Channel protocol combines the authenticated key establishment with authenticated encryption to provide confidentiality, integrity, authenticity, and replay protection. During the handshake, X25519 establishes a shared secret, HMAC authenticates both parties using a preshared key and HKDF derives independent session keys and nonce material. During secure messaging, ChaCha - Poly1305 protects every message while authenticating both the encrypted payload and the associated plaintext header. Strictly increasing sequence numbers and deterministic nonce construction ensure that replayed or reordered messages are detected and rejected. Developing each primitive from its formal specification provided practical experience with how modern cryptographic protocols are constructed and why each component is necessary. The project reinforced the importance of correct algorithm implementation, careful protocol composition, secure key management, authenticated encryption, and rigorous testing against official standards. Together, these elements produce a secure communication channel that reflects the fundamental design principles used by contemporary protocols such as TLS 1.3, Signal, SSH, and WireGuard.